

spinalSynth: Filter Implementation Specification

This document details the hardware implementation specifications for the State Variable Filter (SVF) subpackage of **spinalSynth**.

Table of Contents

- 1. SVF (Top-Level Wrapper)
- 2. ParameterMapper
- 3. FilterCore
- 4. FilterMux
- 5. Register Map Integration
- 6. Signal Flow & Pipeline Timing

1. SVF (Top-Level Wrapper)

Purpose

The SVF top-level wrapper acts as the coordinator. It encapsulates the three submodules (**ParameterMapper**, **FilterCore**, and **FilterMux**), binds them together, registers parameters to the global register map, and coordinates the clock-gated flow of audio samples.

IO Bundle

```
class SVF extends Component {
  val io = new Bundle {
    val phaseTick  = in Bool()                // 480 kHz sample rate tick
    val config      = in(FilterConfig())       // Unified register configuration bundle

    val sampleIn    = slave(Flow(SInt(16 bits))) // 16-bit audio input
    val sampleOut   = master(Flow(SInt(16 bits))) // 16-bit audio output
  }
}
```

Internal Architecture

The top-level wrapper instantiates the submodules, handles parameter conversion, and drives the state variables. When **enable** is deasserted, it clears the internal filter state registers by asserting the **clear** line of **FilterCore** and routes a constant 0 to the output payload, whilst keeping the output flow's **valid** signal pulsing synchronously with the input flow's **valid** (which is gated by **phaseTick**).

2. ParameterMapper

Purpose

The **ParameterMapper** converts user-facing 8-bit parameters into high-precision coefficients used by the Chamberlin SVF equations.

IO Bundle

```
class ParameterMapper extends Component {
  val io = new Bundle {
    val cutoff      = in UInt(8 bits)
    val resonance   = in UInt(8 bits)
    val cutoffCoeff = out UInt(12 bits)
    val resonanceCoeff = out UInt(8 bits)
  }
}
```

Mapping Equations & ROM Tables

- **Cutoff ROM (256 x 12 bits):** Maps the input exponentially to generate a log-like frequency distribution.

```
cutoffCoeff = round(10.0 * (4095.0 / 10.0)^(p / 255.0))
```

- **Resonance ROM (256 x 8 bits):** Maps the input quadratically, where higher input values represent higher resonance (lower damping coefficient).

```
resonanceCoeff = round(255.0 - 251.0 * (r / 255.0)^2)
```

3. FilterCore

Purpose

The **FilterCore** implements the core Chamberlin fixed-point arithmetic loops. Rather than implementing parallel arithmetic logic, it uses a **time-multiplexed shared architecture** with exactly **one multiplier** and **one adder/subtractor** to save hardware resource area. The calculation steps are scheduled over multiple clock cycles within the 50-cycle sample frame budget.

IO Bundle

```
class FilterCore extends Component {
  val io = new Bundle {
    val phaseTick      = in Bool()
    val clear          = in Bool()           // Clears states to 0 when filter is
disabled
    val sampleIn       = in SInt(16 bits)
    val cutoffCoeff     = in UInt(12 bits)
    val resonanceCoeff = in UInt(8 bits)

    val lp             = out SInt(24 bits)
    val bp             = out SInt(24 bits)
    val hp             = out SInt(24 bits)
    val done           = out Bool()         // Pulses High when calculation
completes
  }
}
```

Shared Arithmetic Operators

To optimize resource usage:

- **Shared Multiplier:** Performs signed multiplication. Inputs are selected via multiplexers depending on the FSM state. Unsigned coefficients are zero-extended to signed values (`intoSInt`) before multiplication.
- **Shared Adder/Subtractor:** Performs signed addition and subtraction. Inputs are selected via multiplexers depending on the FSM state.

FSM State Scheduling

The sequencing of arithmetic operations is managed using a `SpinalHDL StateMachine` from the `spinal.lib.fsm` library. The FSM cycles through 8 states to compute the complete SVF response:

State	Step / Operation	Multiplier Inputs	Adder Inputs	Registers Updated
IDLE	Wait for <code>phaseTick</code>	-	-	Latch <code>sampleIn</code> on trigger
CALC_RES	<code>resTerm = (bp * resonanceCoeff) >> 8</code>	<code>bp</code> , <code>resonanceCoeff.intoSInt</code>	-	<code>resTerm</code> (24b)
SUB_INPUT	<code>tempSub = input - lp</code>	-	<code>inputExt</code> , <code>lp</code> (Sub)	<code>tempSub</code> (24b)
CALC_HP	<code>hp = tempSub - resTerm</code>	-	<code>tempSub</code> , <code>resTerm</code> (Sub)	<code>hp</code> (24b)
CALC_BP_TERM	<code>bpTerm = (hp * cutoffCoeff) >> 12</code>	<code>hp</code> , <code>cutoffCoeff.intoSInt</code>	-	<code>bpTerm</code> (24b)
UPDATE_BP	<code>bpNext = bp + bpTerm</code>	-	<code>bp</code> , <code>bpTerm</code> (Add)	<code>bp</code> state register
CALC_LP_TERM	<code>lpTerm = (bpNext * cutoffCoeff) >> 12</code>	<code>bpNext</code> , <code>cutoffCoeff.intoSInt</code>	-	<code>lpTerm</code> (24b)
UPDATE_LP	<code>lpNext = lp + lpTerm</code>	-	<code>lp</code> , <code>lpTerm</code> (Add)	<code>lp</code> state register, done

When `clear` is asserted, FSM resets to `IDLE` and internal state registers (`lp`, `bp`) are cleared to 0.

4. FilterMux

Purpose

The `FilterMux` selects the appropriate response based on the filter mode and scales/resizes the internal 24-bit representation back to the 16-bit output using saturating logic.

IO Bundle

```
class FilterMux extends Component {
  val io = new Bundle {
    val mode      = in UInt(2 bits)
    val lp        = in SInt(24 bits)
    val bp        = in SInt(24 bits)
    val hp        = in SInt(24 bits)
    val sampleOut = out SInt(16 bits)
  }
}
```

Downsizing with Saturation Clamping

Rather than using simple MSB truncation (which leads to severe wrap-around noise when the internal state variables exceed 16-bit limits), `FilterMux` implements a saturating clamp to bound the output values within the signed 16-bit range:

```
sampleOut = {
  case selected > 32767 => 32767
  case selected < -32768 => -32768
  case _ => selected.resize(16 bits)
}
```

This preserves the internal 8-bit headroom in the 24-bit FSM registers (`lpReg`, `bpReg`) to keep the filter loop stable, while ensuring output signal peaks clip cleanly (similar to analog saturation) rather than wrapping around.

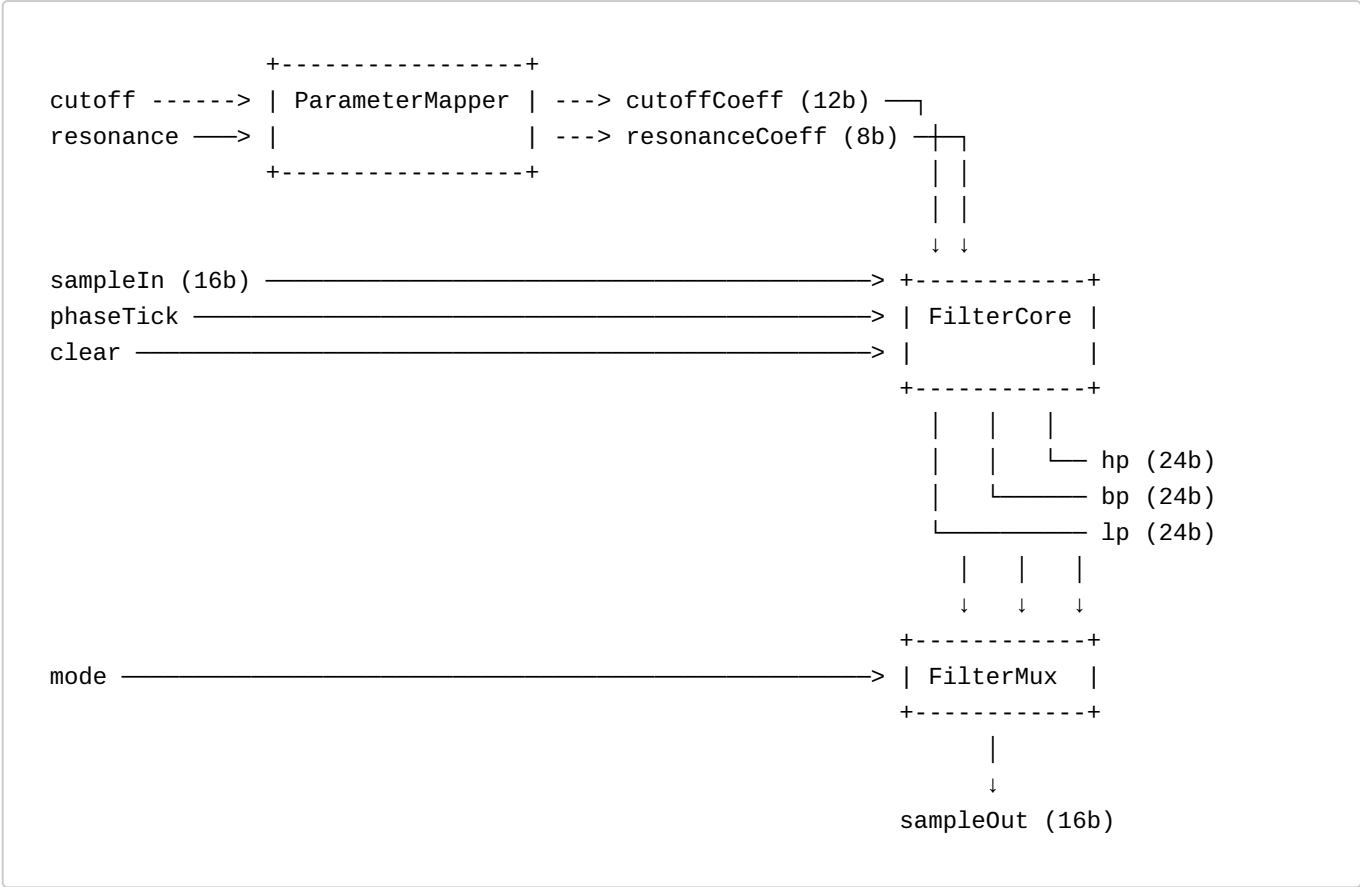
5. Register Map Integration

The register space for the Filter Module is allocated at addresses 0x50 to 0x53:

Register Address	Name	Description
0x50	FILTER_ENABLE	Bit 0: Filter Enable (0 = disabled, 1 = enabled)
0x51	FILTER_MODE	Bits 1:0: Response Mode (00 = LP, 01 = BP, 10 = HP, 11 = Reserved)
0x52	FILTER_CUTOFF	8-bit user-facing cutoff frequency
0x53	FILTER_RESONANCE	8-bit user-facing resonance / feedback

The `RegisterBank` will commit updates to these registers on the main clock domain, and outputs will be routed through a `RegNext` synchronization stage to the SVF top-level wrapper inputs.

6. Signal Flow & Pipeline Timing



Latency

The filter operations are executed sequentially by the FSM. Once calculations are complete, the resulting sample is held in an intermediate output register.

On the next incoming `phaseTick` pulse, the top-level wrapper asserts the output `sampleOut.valid` and drives the payload. This introduces a latency of exactly **1 sample period** (50 system clock cycles) between the input and output flows. Since the output `valid` is aligned to the 480 kHz `phaseTick` grid, it eliminates fractional clock-cycle offsets and allows clean downsampling downstream.